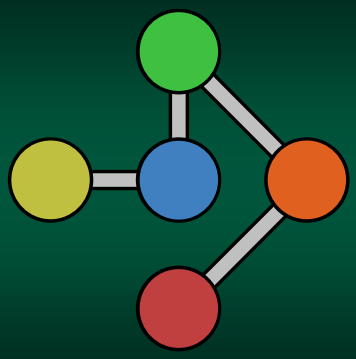
Graph Spanning Trees

Part 15

1



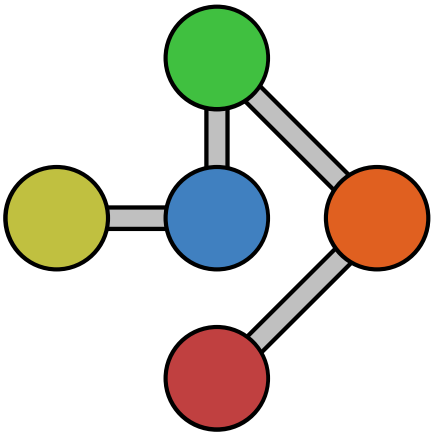
Spanning Trees

Connect the Dots (la la la la)

2

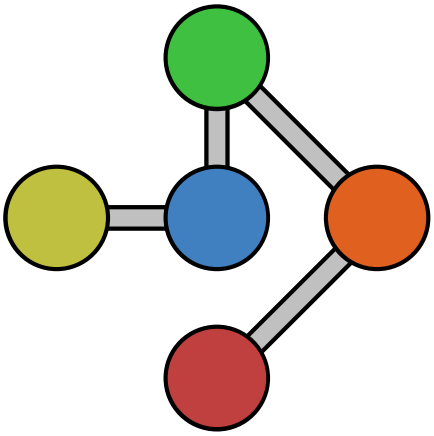
Spanning Trees

- In many cases, we want to make sure that every vertex is connected in a graph
- A *Spanning Tree* includes every vertex of the original graph (an no cycles, obviously)



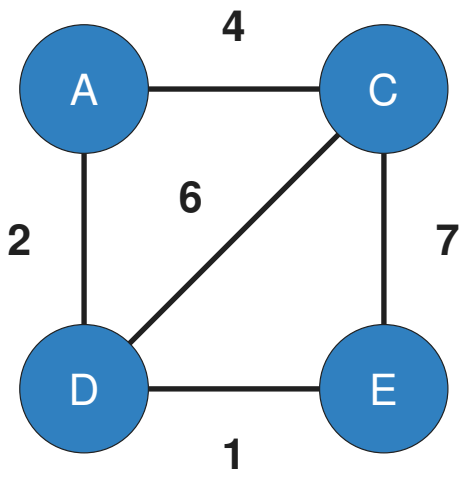
Why Spanning Trees?

- Often, we want to make sure all vertices are connected
- These can be anything from cities (connected by roads) to computers (connected by cables)
- ... just to name a few…



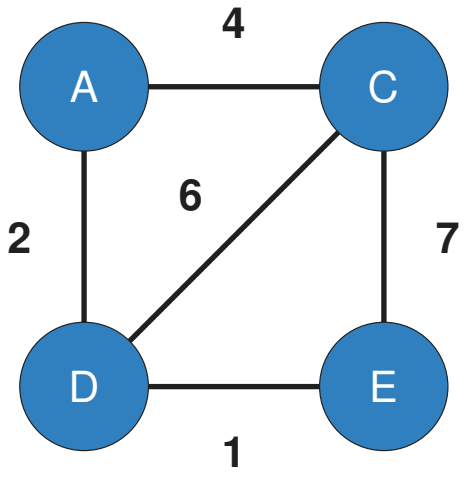
Example Graph

- The graph on the left is quite simple – containing just 4 vertices
- We need to create a tree that contains all 4 vertices

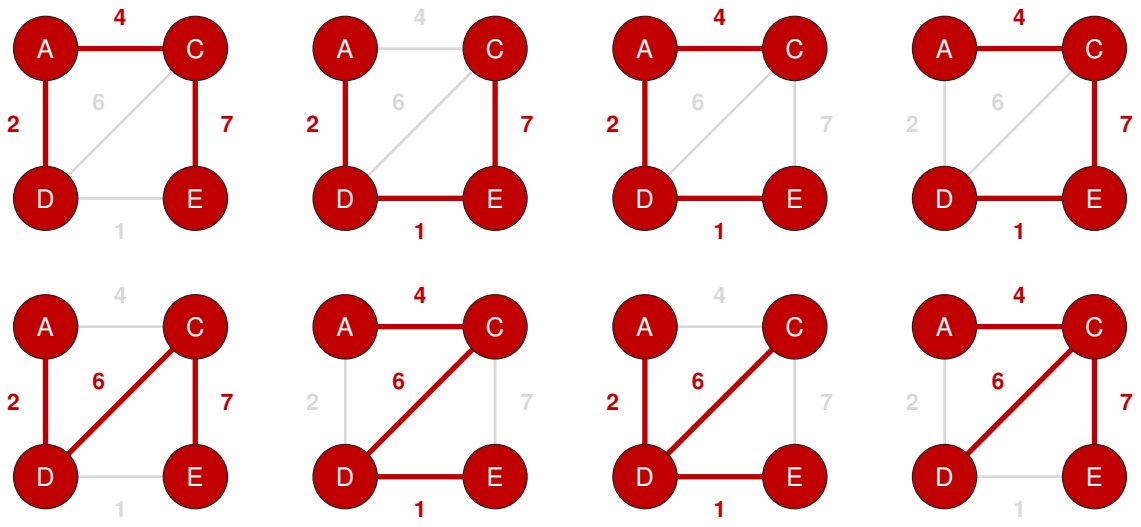


Example Graph

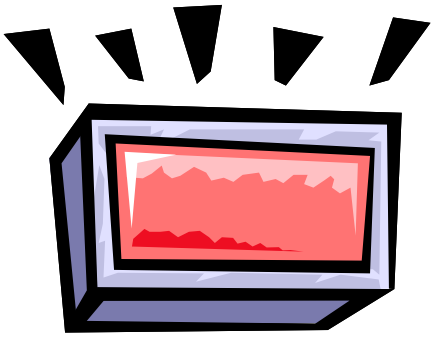
- As you can imagine, most graphs have multiple solutions
- Also note that this graph is weighted
- This can make some solutions better than others



Example: Spanning Trees

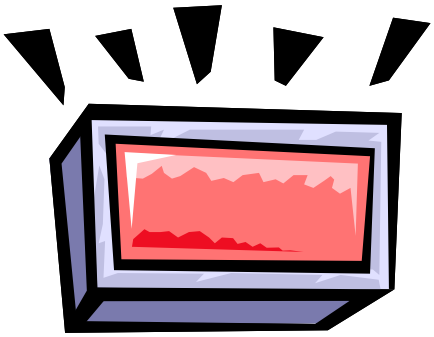


So How Many Spanning Trees?



- Let's look at the most complex graph possible – a complete one
- In this one, every vertex is connected to every other vertex
- *This results in a **large** number of possible solutions*

So How Many Spanning Trees?

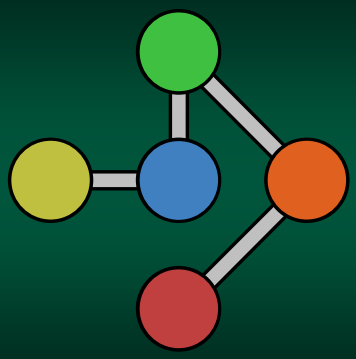


- Given a complete graph of *N* vertices, there are N^{N-2} possible spanning trees
- Yes, that is N to the power of N – which is even ***worst*** than exponential growth!

Complete Graph Spanning Trees

Vertices	Total Spanning Trees
1	1
2	1
3	3
4	16
5	125
6	1,296

Vertices	Total Spanning Trees
7	16,807
8	262,144
9	4,782,969
10	100,000,000
11	2,357,947,691
12	61,917,364,224



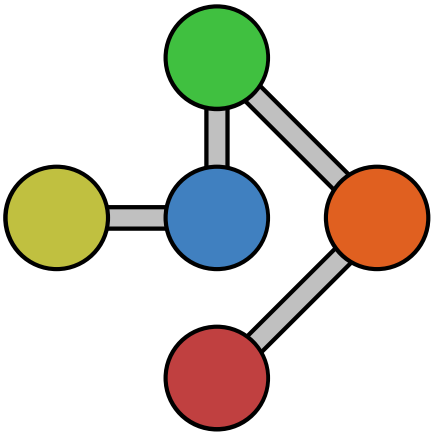
Minimum Spanning Tree

Getting the "Best" Tree

11

Minimum Spanning Tree

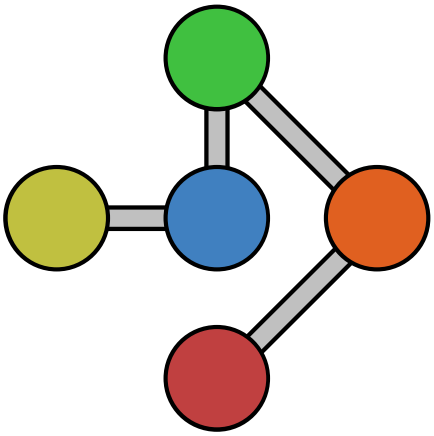
- *Minimum Spanning Tree (MST)* contains all the vertices of a graph with the minimum possible weight
- Naturally, depending on the weights, some solutions are more desirable than others



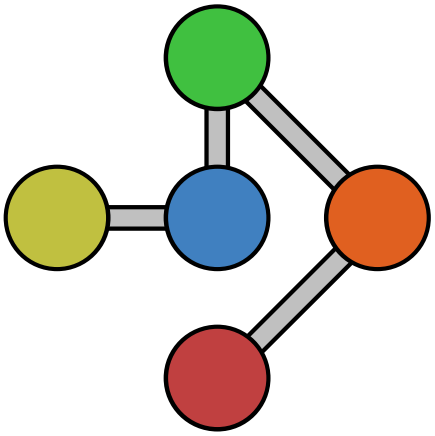
12

Minimum Spanning Tree

- Some uses (minimize cost)
  - building cable networks
  - building a road that joins cities
- So, creating an algorithm – that computes these trees - *is of vital importance*

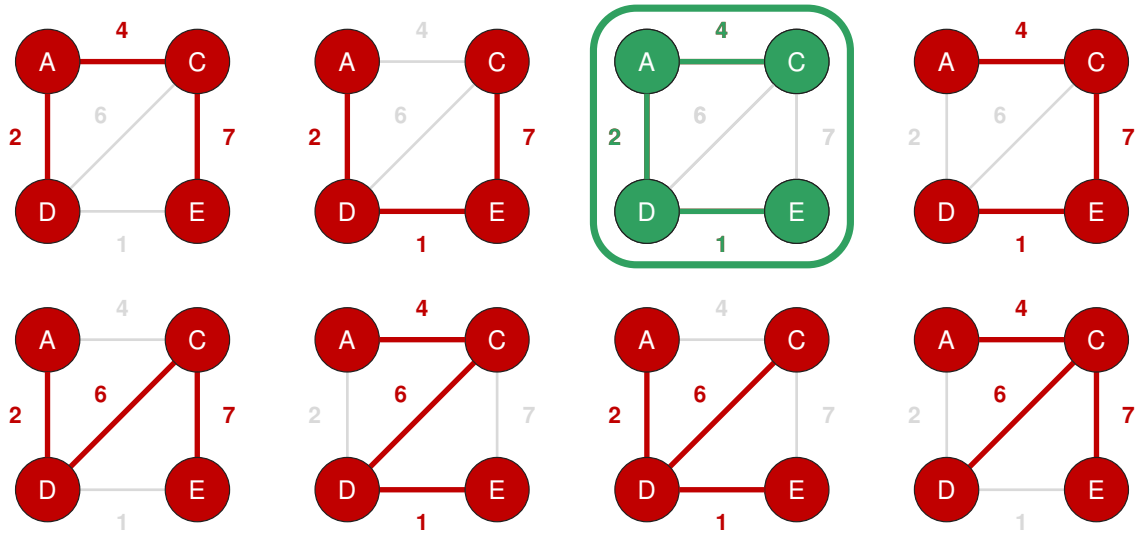


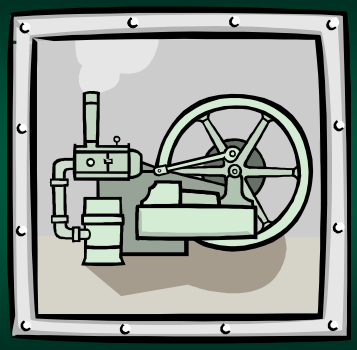
Uniformly Weighted Graph



- If each edge has the same weight, then the computation of the minimum spanning tree is trivial
- A breadth-first or depth-first search can be used
- …but many are weighted

Example: Minimal Spanning Tree



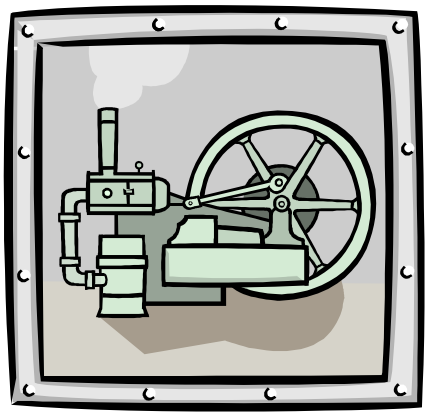


Brute Force

Doing in the hard way
(and impossible)

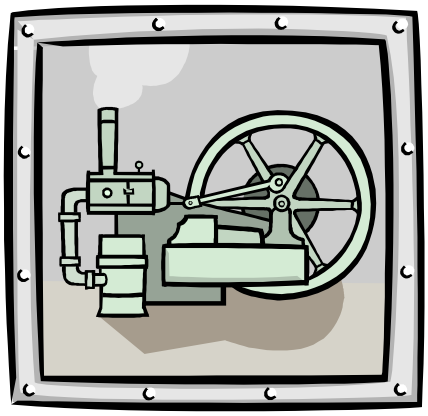
Brute Force

- One way to compute the minimum spanning tree is to simply ***try all of them!***
- Approach:
  - calculate every tree
  - keep track of the tree with the minimum total weight



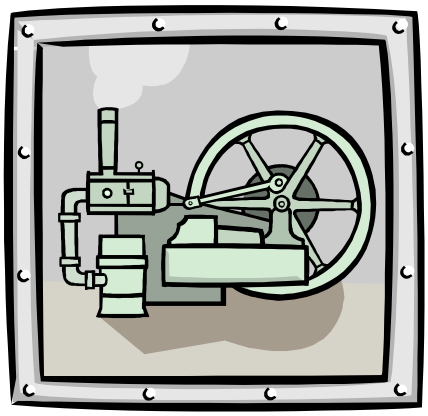
Brute Force

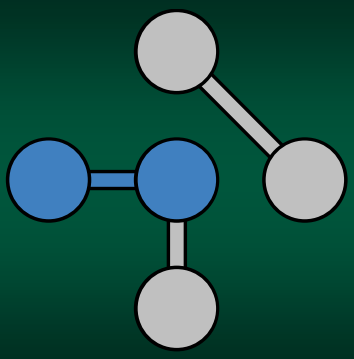
- Calculating <u>just one tree</u> will require O(n) time where *n* is the total number of vertices
- How many trees are there?
- For complete graphs, we know it's n^{n-2} – which is what we must use for Big-O



Brute Force

- So, there is a $n \times n^{n-2}$ computational requirement
- Which is **$O(n^{n-1})$**
- Naturally, this is a poor – and ***impossible*** – solution



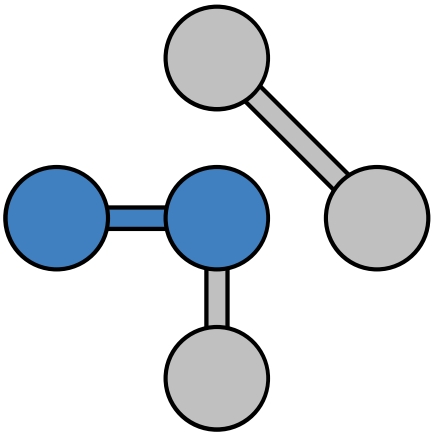


Kruskal's Algorithm

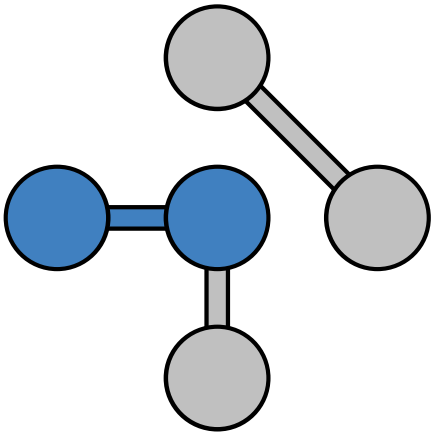
Computing a Minimum Spanning Tree

Kruskal's Algorithm

- *Kruskal's Algorithm* computes a minimum spanning tree
- It was invented by *Joseph Kruskal* in 1956 *(8 BBW)*
- Published in *Proceedings of the American Mathematical Society*

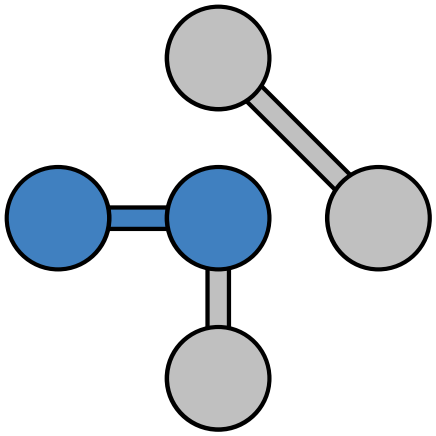


Kruskal's Algorithm



- It conceptually starts with a forest of single vertices
- It then adds edges – with the minimum weight – to connect the two vertices (and possible subtrees)

Kruskal's Algorithm



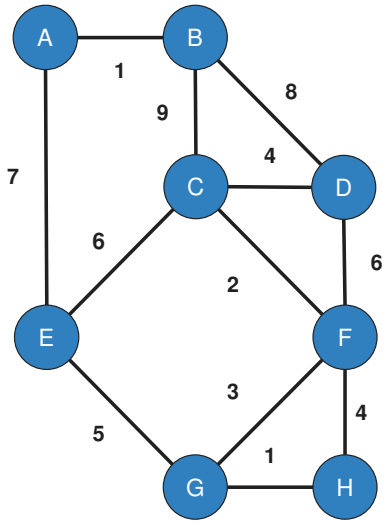
- So, it adds edges in sorted order of weight
- If an edge would cause a cycle, then it is rejected and not part of the solution
- This is avoided by using a **union-find** set to merge subtrees

Kruskal's Algorithm

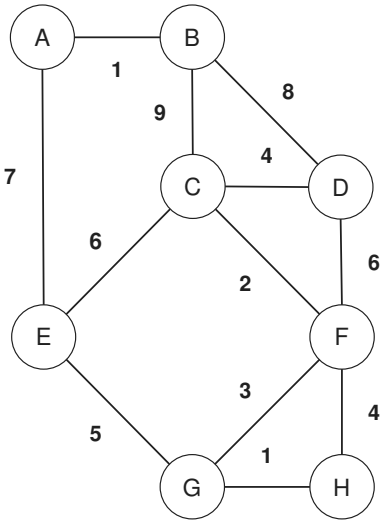
```
Create a list of edges sorted by weight
Add each vertex to a Union-Find
loop through the list
   if the vertices have different representatives
      Add the edge
      Union the two vertices (in the Union-Find)
   end if
end loop
```

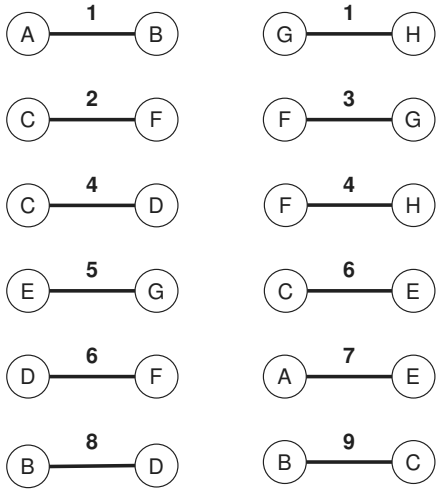
Kruskal's Algorithm Example

- The following is a graph with multiple weights
- Kruskal's Algorithm will create a list of the edges sorted by weight

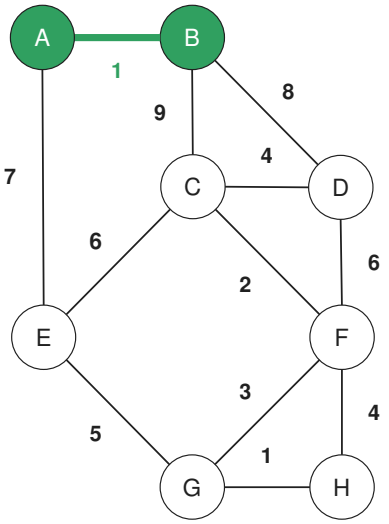


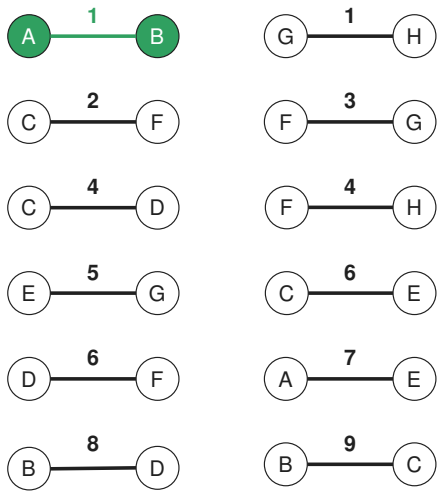
Kruskal's Algorithm Example



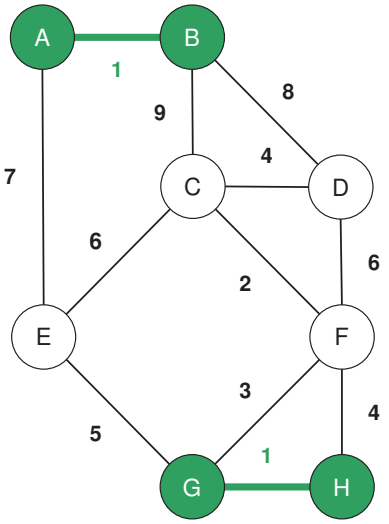


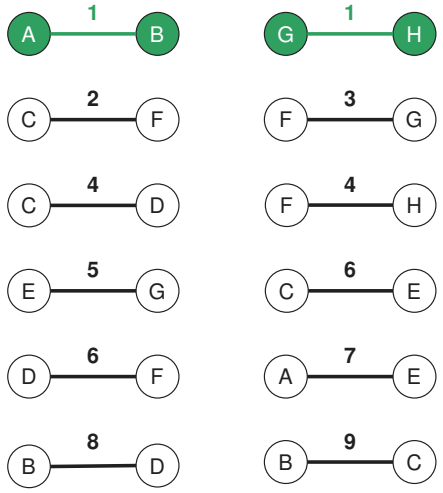
Add Edge



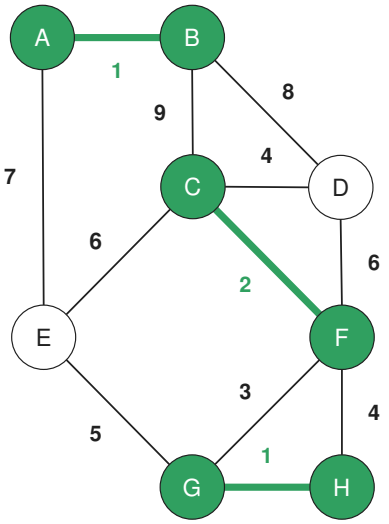


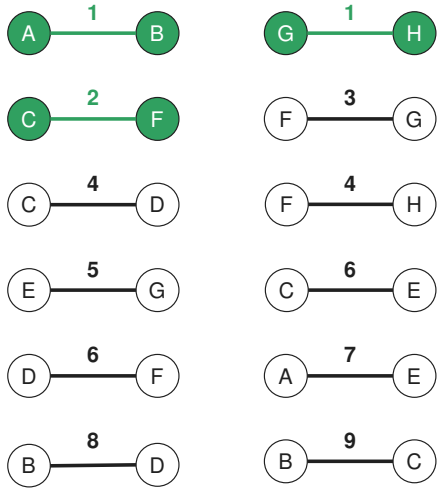
Add Edge



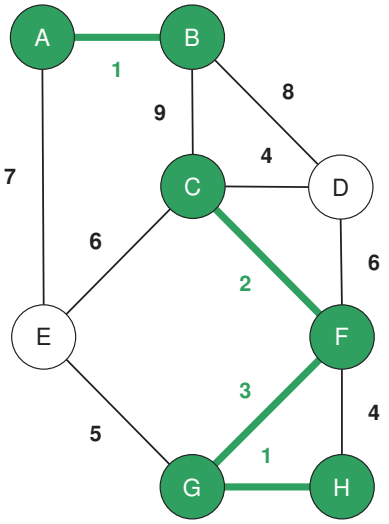


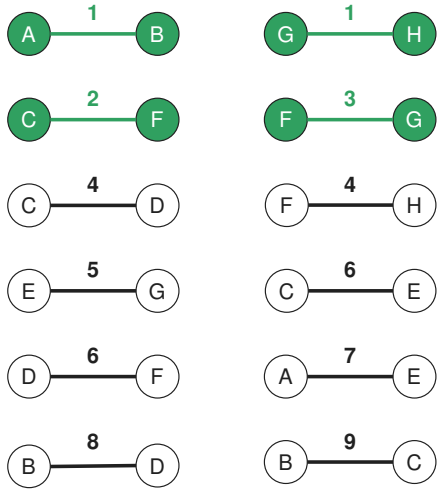
Add Edge



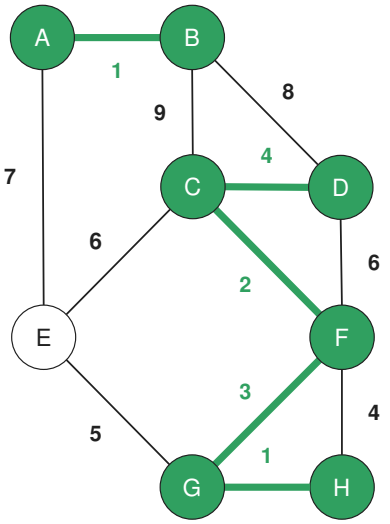


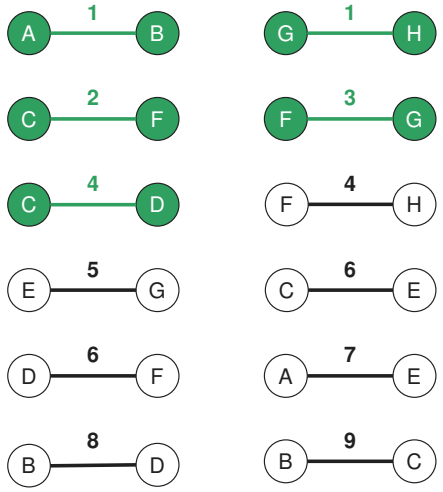
Add Edge



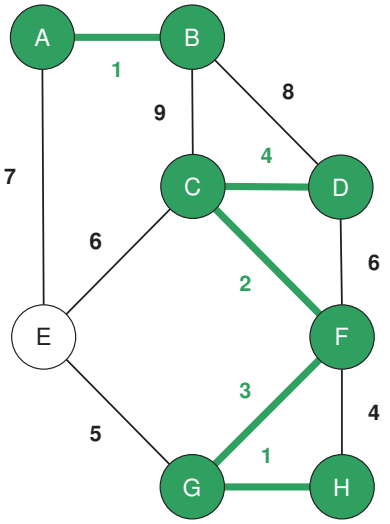


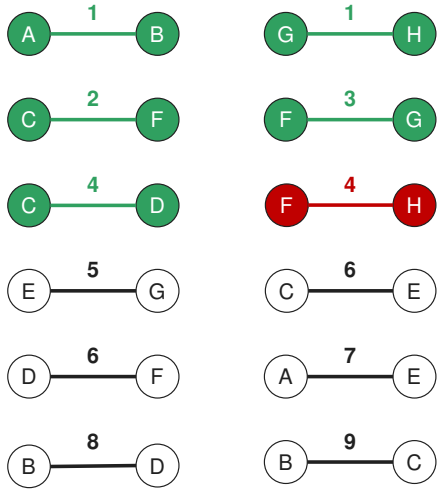
Add Edge



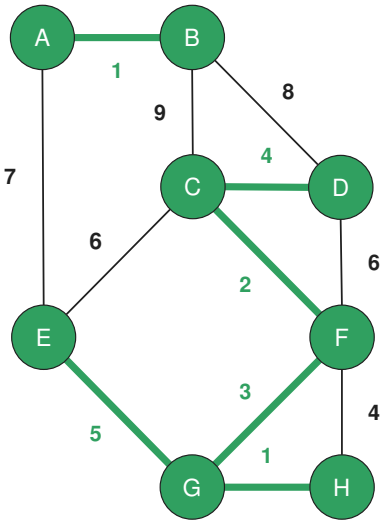


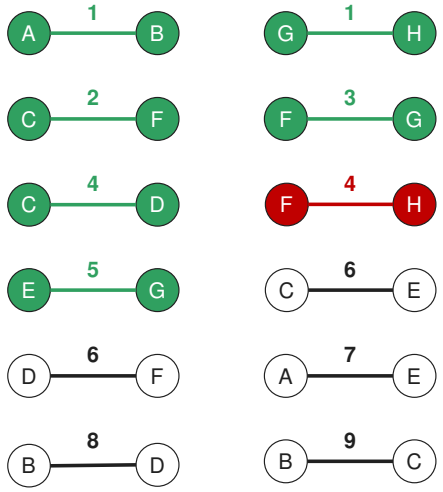
Reject: It would create a cycle



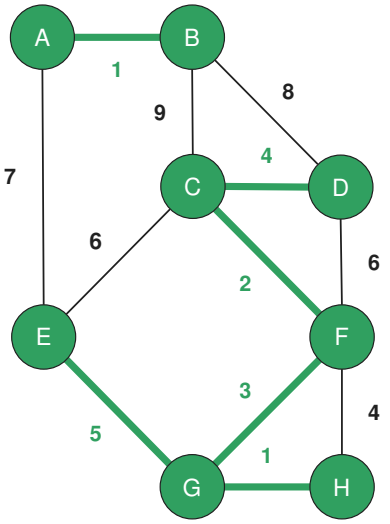


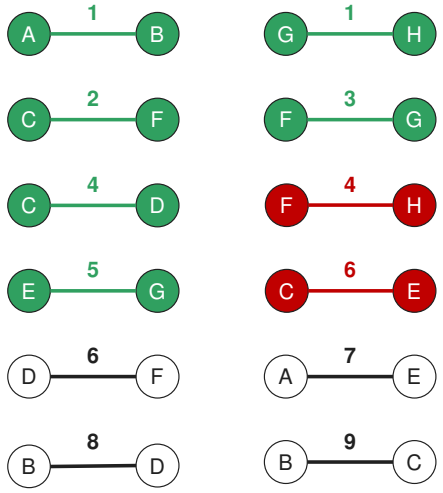
Add Edge



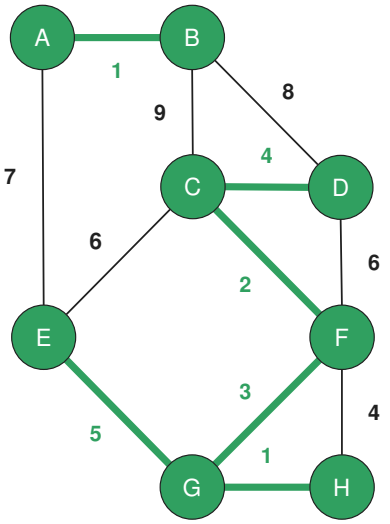


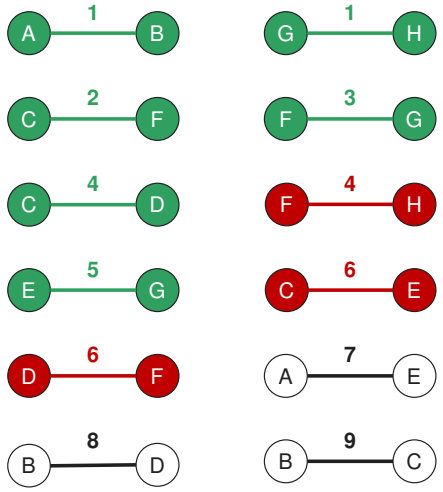
Reject: It would create a cycle



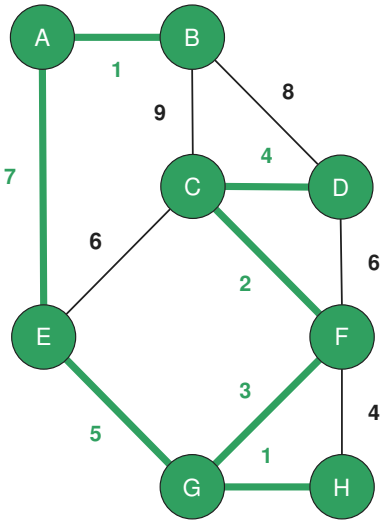


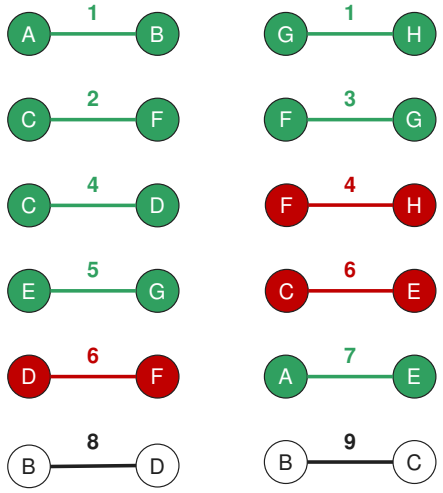
Reject: It would create a cycle



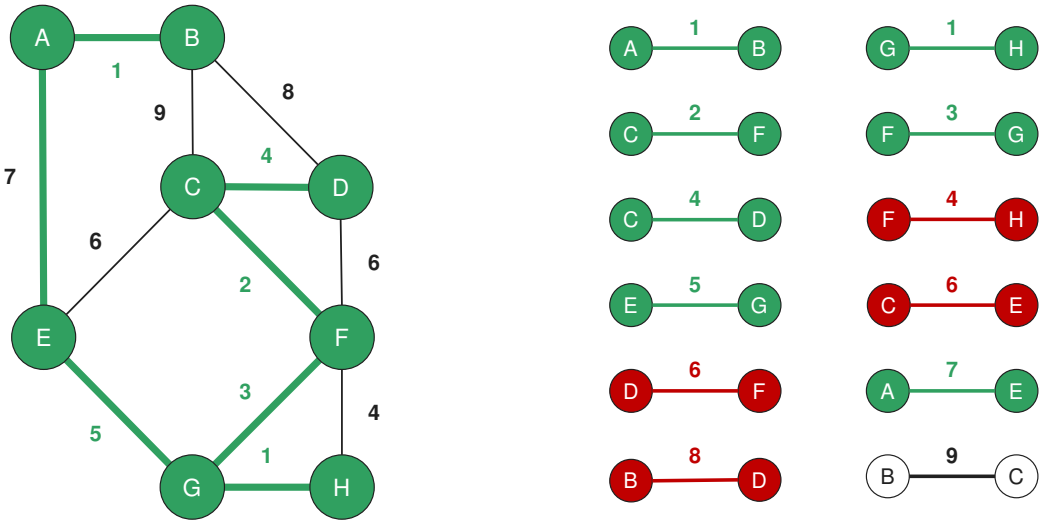


Add Edge

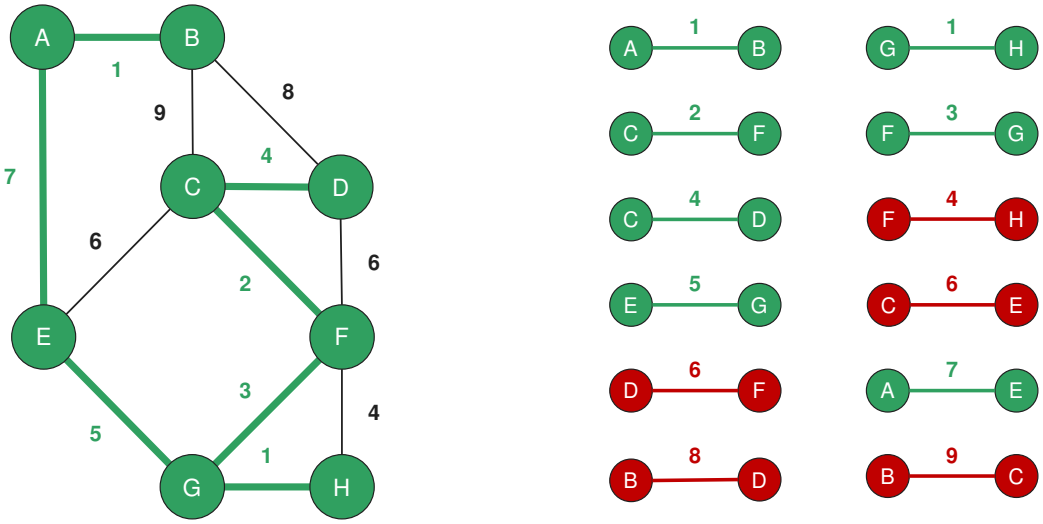




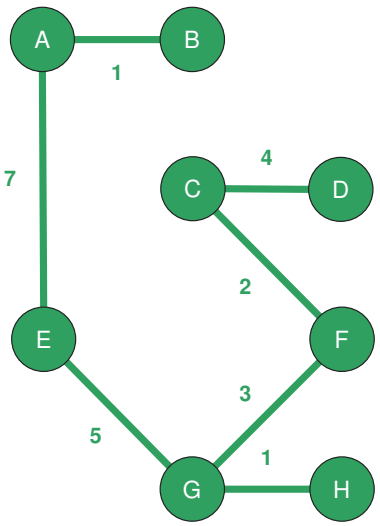
Reject: It would create a cycle

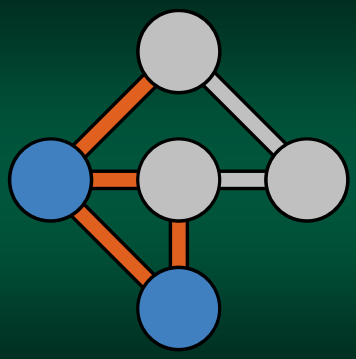


Reject: It would create a cycle



Resulting Minimum Spanning Tree



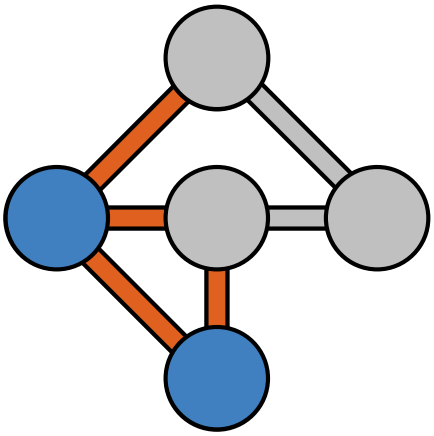


Prim-Jarník Algorithm

Computing a Minimum Spanning Tree

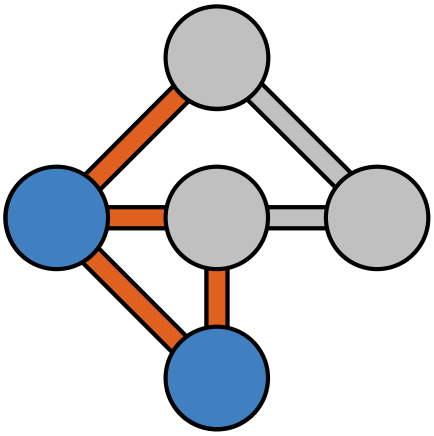
Prim-Jarník Algorithm

- *Prim's Algorithm* computes a minimum spanning tree
- It was invented by *Robert C. Prim* in 1956 *(8 BBW)* … same year as Kruskal

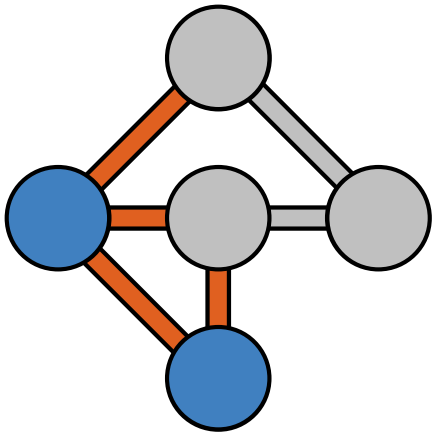


Prim-Jarník Algorithm

- However, it was discovered that computer scientist *Vojtěch Jarník* had also invented it in 1930 *(34 BBW)*
- So, it is sometimes called the *Prim-Jarník Algorithm*

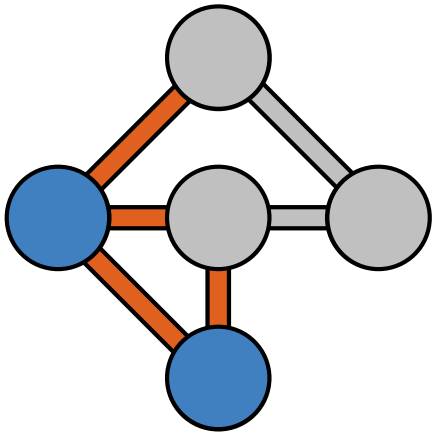


Prim-Jarník Algorithm



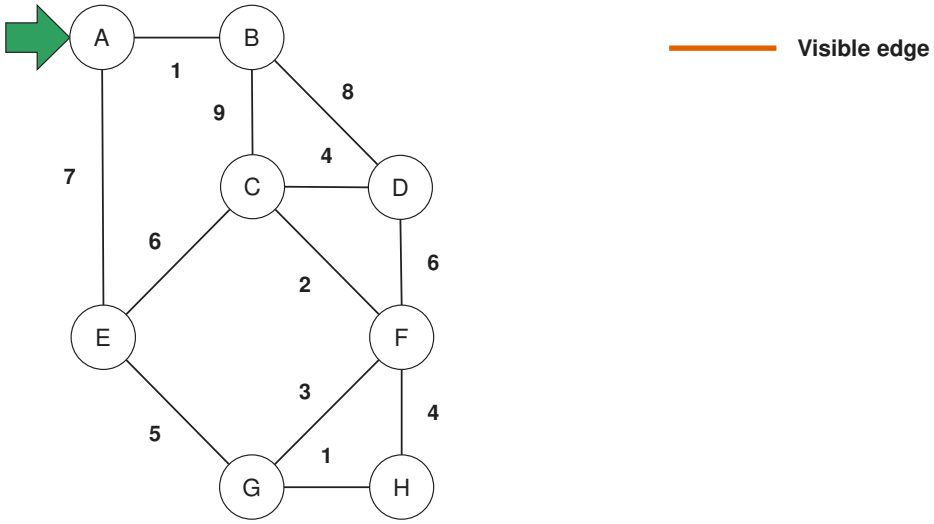
- This algorithm starts with an *arbitrarily chosen*, vertex
- The algorithm then adds the minimal edge – that the entire current tree can currently "see"

Prim-Jarník Algorithm

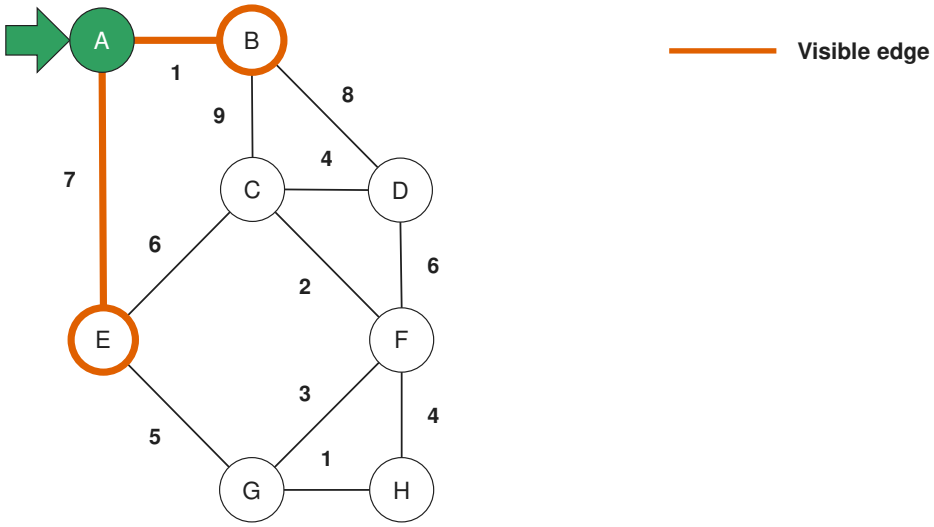


- So, it checks all the edges connect to unvisited vertices
- The algorithm is *greedy* – always following the local optimal path – to grow the tree

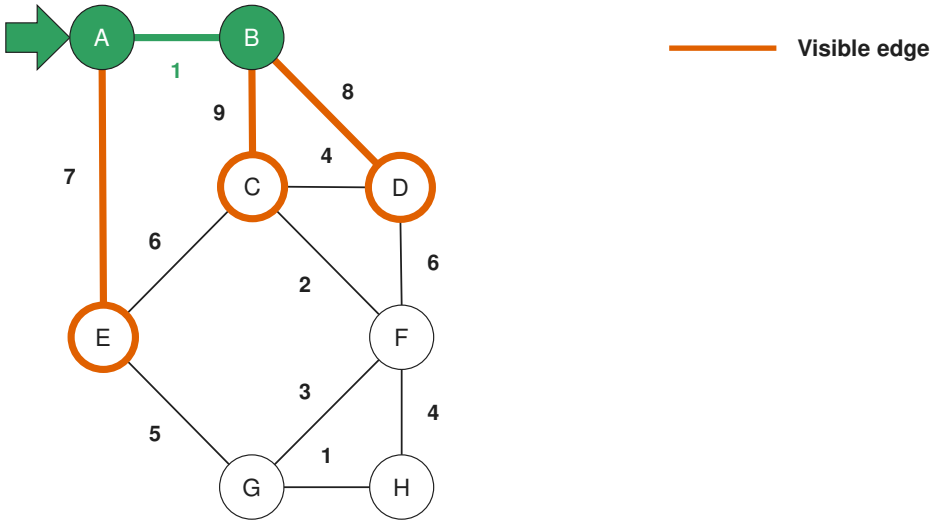
Prim-Jarník Algorithm Example



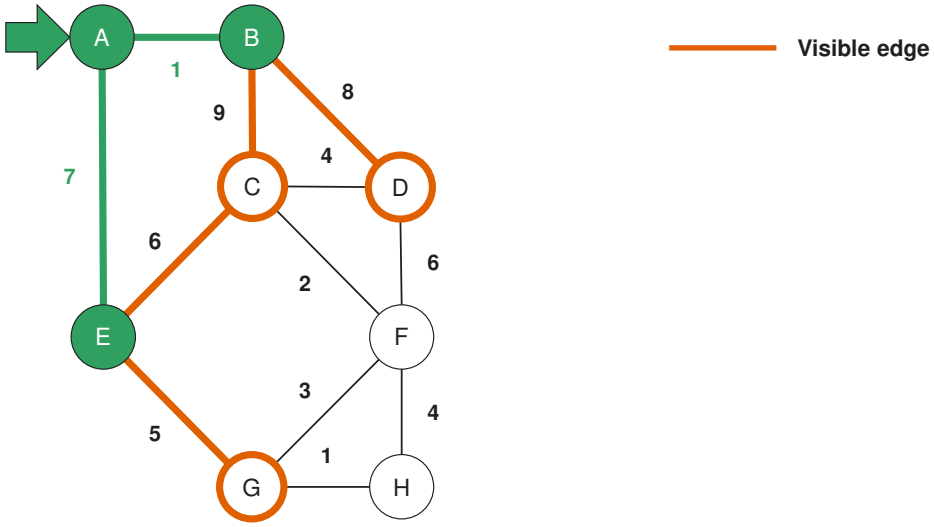
Prim-Jarník Algorithm Example



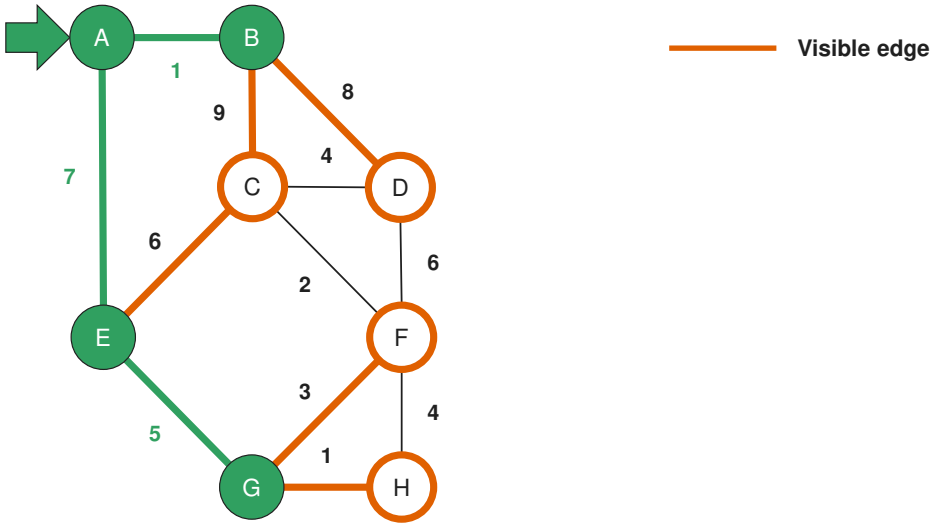
Prim-Jarník Algorithm Example



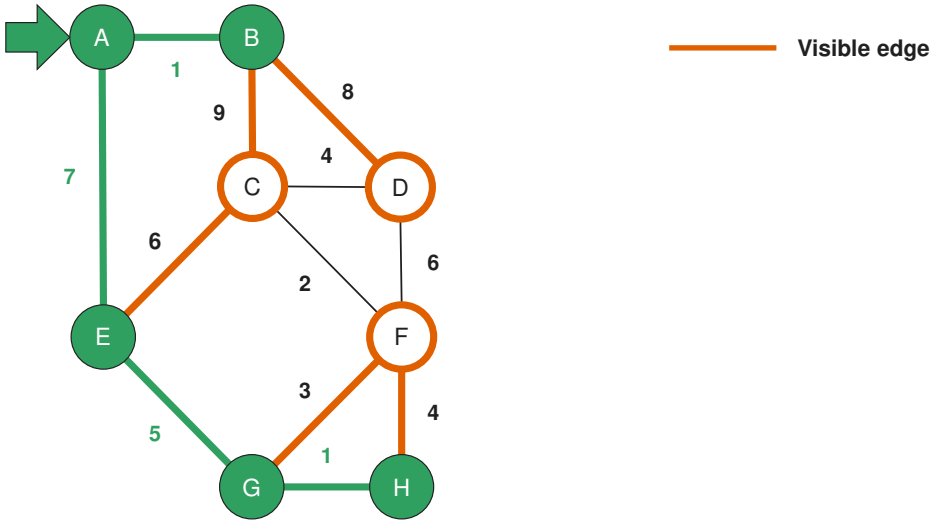
Prim-Jarník Algorithm Example



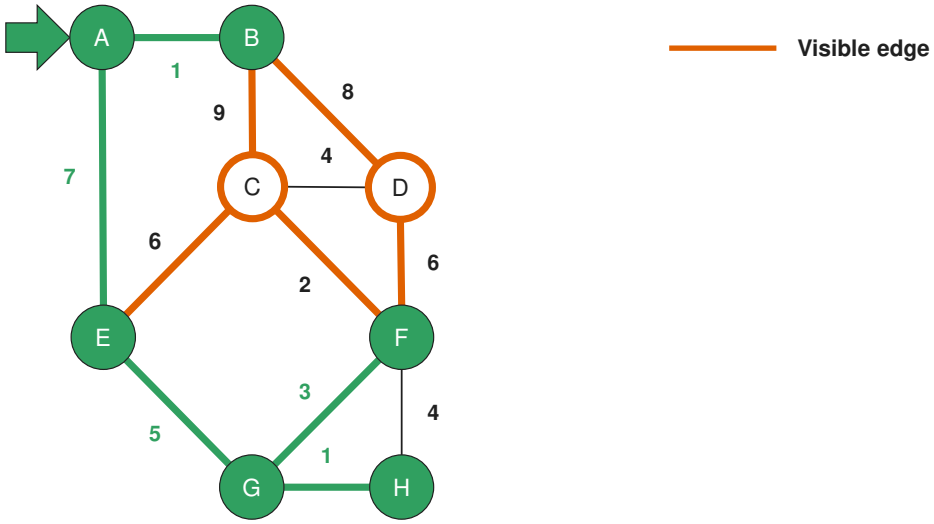
Prim-Jarník Algorithm Example



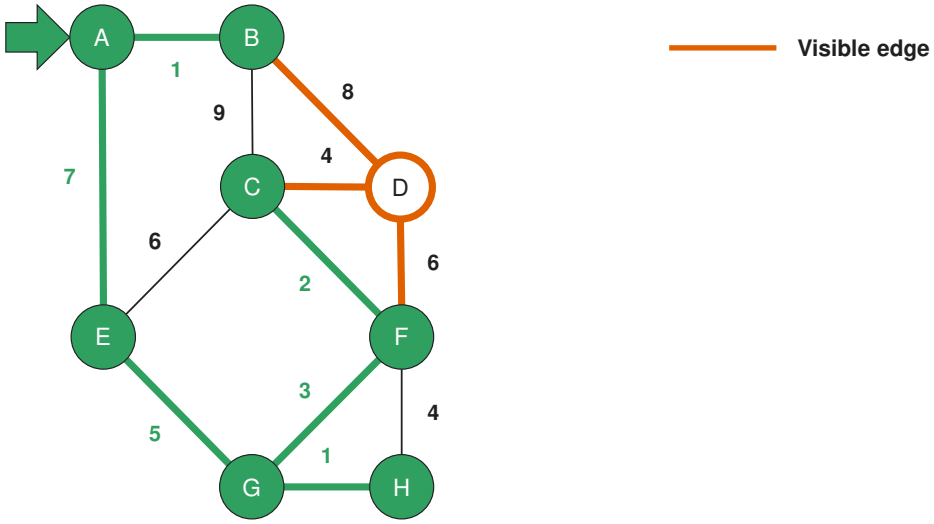
Prim-Jarník Algorithm Example



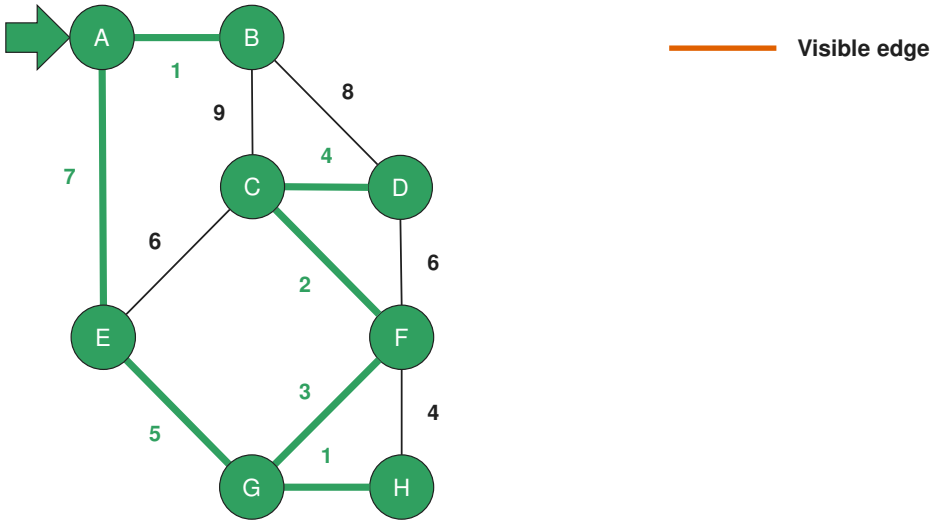
Prim-Jarník Algorithm Example



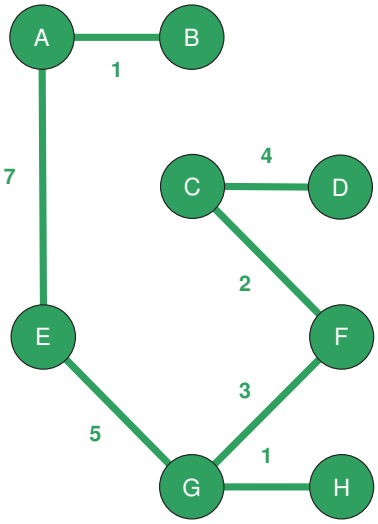
Prim-Jarník Algorithm Example



Prim-Jarník Algorithm Example



Resulting Minimum Spanning Tree



Kruskal vs. Prim-Jarník

- Both algorithms find the minimum spanning tree
  - *though not necessarily identical*
  - *… if multiple, equal, solutions exist (they might)*
- Both are O(|E| log |V|) where |E| is the number of edges and |V| is the number of vertices